

Action required

Action required

1. Item: Partial answer handling within a collection round

Context: In PoC, each user turn closes the current round. If the system asks for name + age + gender and the user sends only "Ravi", the round closes with only name extracted. Remaining fields (age, gender) are treated as missing and targeted in the next available round.

Agent Core — Implementation Design

Agent Core — Implementation Design

Folder structure

```
agent_core/
├── __init__.py
├── models.py           # Shared data models (inputs, outputs, state)
├── exceptions.py       # Typed exceptions
├── base.py             # AgentCoreBase — public contract
├── orchestrator.py     # Concrete AgentCore — turn execution
├── llm_wrapper.py      # LLM interface + Claude implementation
├── tool_registry.py    # Tool definition manager
├── manager_agent.py    # Tool-use loop + consent gate
├── interfaces/
│   ├── memory_layer.py
│   ├── trust_layer.py
│   ├── action_gateway.py
│   ├── knowledge_engine.py
│   ├── reach_layer.py
│   └── learning_layer.py
├── tests/
│   ├── test_orchestrator.py
│   ├── test_llm_wrapper.py
│   ├── test_manager_agent.py
│   └── test_tool_registry.py
```

Components

models.py Shared dataclasses — TurnInput, SessionState, TrustCheckResult, ToolCall, ToolResult, LLMResponse, TurnResult, TurnEvent

exceptions.py Typed exceptions — LLMCallError, LLMFallbackError, TrustViolationError, ToolExecutionError, ConsentRequiredError, ConfigurationError

base.py AgentCoreBase ABC
 └─ process_turn(TurnInput) → TurnResult

orchestrator.py AgentCore — implements the 10-step turn sequence
 Enforces both Trust Layer checks
 Schedules async steps after response is returned

llm_wrapper.py LLMWrapperBase ABC + ClaudeLLMWrapper
Only file that calls the Anthropic API
Owns retry logic and fallback model switching

tool_registry.py ToolRegistry — loads tool definitions from config at startup
Validates definitions, exposes them to every LLM call
Knows which tools require consent (write/identity type)

manager_agent.py ManagerAgent — drives the LLM → tool → LLM loop
Enforces consent check before write/identity tool execution

Turn Execution Sequence

① Read session state → MemoryLayer
② Check input → TrustLayer (block/escalate exits here)
③ Assemble prompt → KnowledgeEngine
④ LLM call #1 → LLMWrapper
⑤ Tool-use loop (if needed) → ManagerAgent → ActionGateway → LLMWrapper
⑥ Check output → TrustLayer (block replaces response with fallback)
⑦ Return TurnResult ← response delivered to caller

— async (daemon thread)

⑧ Write updated state → MemoryLayer
⑨ Emit turn event → LearningLayer

Rules

Only LLM caller llm_wrapper.py — no other file imports Anthropic SDK

Trust Layer runs twice orchestrator.py — both check_input and check_output
are hardcoded in process_turn; neither is skippable

Async steps never block orchestrator.py — steps ⑧⑨ run in daemon thread
after TurnResult is returned

Write connectors need manager_agent.py — consent checked before every

consent write/identity tool call via tool_registry + trust_layer

Stateless between turns orchestrator.py holds no instance state
All state loaded fresh from MemoryLayer each turn

Config drives behaviour llm_wrapper, tool_registry, orchestrator all read
from injected DomainConfig — nothing hardcoded

No internal coupling All DPG dependencies injected as base class interfaces
at construction; no concrete imports across modules

Implementation Order

Phase 1 models.py, exceptions.py No dependencies; foundation for everything

Phase 2 interfaces/*.py (all 6) ABCs only; define contracts before any logic

Phase 3 llm_wrapper.py, tool_registry.py Core utilities; no inter-dependency between them

Phase 4 manager_agent.py Depends on llm_wrapper + tool_registry + 2 interfaces

Phase 5 base.py, orchestrator.py Final wiring; depends on everything above

Phase 6 tests/ Written alongside or after each component

Knowledge Engine - Implementation plan

Knowledge Engine - Implementation plan

Overview

The Knowledge Engine (KE) is a DPG building block responsible for assembling the complete prompt before every LLM call. It runs as a separate service and is called by Agent Core at Step 3 of every conversation turn — after the Trust Layer clears the input and before the main LLM call is made.

KE does not call the Anthropic API directly. All LLM calls within KE go through Agent Core's internal proxy endpoint (POST /internal/llm/call), keeping Agent Core as the sole holder of the Anthropic API key.

What KE Produces

Every call to KE returns a single list[dict] — the complete messages array passed directly to the main LLM. This array contains:

- A system prompt block (persona, language instruction, RAG context, guardrail reminders)
- Conversation history from the current session
- The current user message

Internal Architecture

KE processes every user message through 5 sequential blocks. Each block reads from and writes to a shared KEContext object. The final prompt is assembled after all blocks complete.

The 5 Blocks

NOTE : Block 1 and 2 (Language Normalisation & NLU Processor) moved to agent core after discussion.

Block 1 — Language Normalisation

- Detects the language of the user's input (Hindi, Hinglish, Kannada, English)

- Handles Roman-script Hindi (transliteration)
- Handles code-switching (mixed language input)
- Provider: LLM-native (Haiku model via Agent Core proxy) or Bhashini API (post-PoC stub)
- Output: normalised input text + detected language

Block 2 — NLU Processor

- Classifies user intent from a configured list (e.g. market_truth_query, scheme_query, training_query, apply_now, counsellor_request, pay_range_query)
- Extracts entities: trade, location, distance, income urgency
- Detects sentiment: neutral, positive, distressed, frustrated
- LLM call returns structured JSON; falls back to intent=unknown on parse failure
- Output: intent, entities, sentiment, confidence score

Block 3 — Glossary & Domain Vocabulary

- Pure string matching — no LLM or database call
- Maps colloquial user terms to canonical domain concepts
- Examples: "bijli wala" → trade:electrician, "PMKVY" → scheme:pmkvy, "Hubli" → location:karnataka_north
- Applied to both the normalised input and extracted entity values
- Mappings defined entirely in YAML config — no hardcoding

Block 4 — Static Knowledge Base (RAG)

- Retrieves the most relevant knowledge chunks from ChromaDB vector database
- Uses OpenAI text-embedding-3-small for semantic embeddings
- Filters retrieval by intent (e.g. scheme_query only retrieves scheme documents) and by location entity
- Always injects a special "always_include" chunk regardless of similarity score
- Two modes: ingest (offline, run once before first use) and retrieve (every turn at runtime)
- Documents ingested: labour schemes PDF, trade descriptions PDF, training institutes CSV, bridge income options PDF, ONEST market truth framing markdown

Block 5 — Multimodal Input Handler

- Handles non-text inputs: PDF (extract text via PyMuPDF), image (describe via LLM vision call)
- Audio: out of scope, returns context unchanged with a log message
- Disabled for KKB PoC (voice channel is text-only); can be enabled via config for other deployments

Supporting Components

KEContext (shared data object)

A dataclass passed through all 5 blocks. Created fresh at the start of every `assemble_prompt()` call. Contains: `session_id`, `raw_input`, `normalised_input`, `detected_language`, `intent`, `entities`, `sentiment`, `confidence`, `retrieval_chunks`, `always_include_chunks`, `session_state`. Never exposed to Agent Core.

KnowledgeEngine Orchestrator (`engine.py`)

- Instantiates all enabled blocks at startup from YAML config
- Runs blocks in fixed order on every call
- Builds the final messages list from the enriched KEContext
- Accepts llm: `HttpLLMWrapper` at construction — this is the only LLM client KE uses

KnowledgeBlock Base Class (`base.py`)

- Abstract base class that every block must inherit from
- Defines the `process(context, llm, config) → KEContext` contract
- Blocks must never raise — on failure they log the error and return context unchanged

HttpLLMWrapper (`llm_proxy_client.py`)

- Implements `LLMWrapperBase` interface
- Makes HTTP POST calls to Agent Core's `/internal/llm/call` endpoint
- Proxy URL comes from YAML config — switching environments is a config change only, zero code changes

Ingestion Script (`scripts/ingest.py`)

- Run once offline before the system starts
- Loads all knowledge documents into ChromaDB with metadata tagging

- Not part of the runtime request path

LLM Call Architecture

KE Block (NLU / Language Norm / Image)



HttpLLMWrapper.call(messages, model_override=haiku)



POST /internal/llm/call → Agent Core → Anthropic API (Haiku)

Note : Agent Core holds the Anthropic API key. KE never holds it.
Local dev and production differ only by the proxy URL in config.

Technology Stack

Component	Technology
Vector database	ChromaDB (local, in-process)
Embedding model	OpenAI text-embedding-3-small
LLM for NLU / language norm	Claude Haiku (via Agent Core proxy)
PDF extraction	PyMuPDF (fitz)
HTTP client (LLM proxy)	httpx

File Structure

```
knowledge_engine/
├── src/
│   ├── base.py           — KnowledgeBlock ABC + KEContext dataclass
│   ├── engine.py         — KnowledgeEngine orchestrator
│   ├── llm_proxy_client.py — HttpLLMWrapper
│   └── blocks/
│       ├── language_normalisation.py — Block 1
│       ├── nlu_processor.py — Block 2
│       ├── glossary.py — Block 3
│       ├── static_knowledge_base.py — Block 4
│       └── multimodal_input_handler.py — Block 5
├── scripts/
│   └── ingest.py — offline document ingestion
├── data/
│   ├── labour_schemes.pdf
│   ├── trade_descriptions.pdf
│   ├── training_institutes.csv
│   ├── bridge_income_options.pdf
│   └── onest_market_truth_framing.md
└── tests/
    ├── test_engine.py
    ├── test_language_normalisation.py
    ├── test_nlu_processor.py
    ├── test_glossary.py
    ├── test_static_knowledge_base.py
    └── test_multimodal_input_handler.py
```

Prerequisite (Agent Core)

Before KE can run end-to-end, Agent Core must implement POST `/internal/llm/call` — an HTTP endpoint that receives a messages payload, calls the Anthropic API via its existing LLM wrapper, and returns the LLM response as JSON.

Memory Layer

Memory Layer — Port 8002

Goal: Manage session state across all active conversations.

What to build

src/session_memory.py — InProcessSessionMemory

A thread-safe, in-process dict-based store implementing MemoryLayerBase. For the PoC, state lives in memory only (no database). Uses a threading.Lock for concurrent access safety.

Method	Behaviour
read_session(session_id)	Return the stored dict for that session; return an empty session skeleton if not found
write_session(session_id, state)	Store/overwrite the full session dict (thread-safe)
get_user_profile(session_id)	Return a demo user profile: {trade: electrician, location: hubli, language: hindi} — hardcoded for PoC
write_user_profile(session_id, profile)	Update the user profile in-process only
clear_session(session_id)	Remove the session from the store

Session state shape:

```
{
  "session_id": str,
  "history": list,
  "confirmed_entities": dict,
  "workflow_step": str | None,
  "user_profile": dict
}
```

Config keys read: memory.session_ttl_seconds, memory.max_sessions — documented but not enforced in PoC.

src/server.py — FastAPI app

Endpoint	Method	Purpose
POST /session/read	{session_id} → full session state dict	Load session for Agent Core
POST /session/write	{session_id, state} → {status: ok}	Persist state after turn
GET /profile/{session_id}	→ user profile dict	Agent Core reads profile on request
DELETE /session/{session_id}	→ {status: ok}	Clear session on logout/expiry
GET /health	→ {status: ok}	Liveness probe

Folder structure

```
memory_layer/
├── main.py
├── config/
│   └── config.yaml
├── src/
│   ├── session_memory.py
│   └── server.py
├── tests/
│   └── test_session_memory.py
└── pyproject.toml
```

Trust Layer

Trust Layer — Port 8003

Goal: Enforce safety rules on every input and every output. Never skipped.

What to build

src/guardrails.py — BasicTrustLayer

Implements TrustLayerBase. Rule engine using case-insensitive substring matching. All rules loaded from YAML config at startup — no hardcoded blocked terms in source code.

Method	Behaviour
check_input(session_id, user_message)	Check against blocked_phrases (→ block) then escalation_topics (→ escalate); empty input always passes
check_output(session_id, llm_response)	Check against output_rules.blocked_phrases; empty response always passes
check_consent(session_id, connector_name)	PoC: always returns True; real implementation would check stored consent record

Result shape for all checks: {passed: bool, action: "allow"|"block"|"escalate", reason: str|None}

Config keys read:

1. trust.input_rules.blocked_phrases — list of strings to block in user input
2. trust.input_rules.escalation_topics — list of strings that require human handoff
3. trust.output_rules.blocked_phrases — list of strings to block in LLM output

src/server.py — FastAPI app

Endpoint	Method	Purpose
POST /check/input	{session_id, message} → trust result	Called by Agent Core before every LLM call
POST /check/output	{session_id, response} → trust result	Called by Agent Core after every LLM call
POST /check/consent	{session_id, connector_name} → {granted: bool}	Called before any write/identity connector execution
GET /health	→ {status: ok}	Liveness probe

Fail-open strategy: If the Trust Layer itself errors (network issue, internal exception), all checks return passed=True, action="allow" rather than blocking the user. This is a PoC policy — a production trust layer would fail-closed.

Folder structure

```
trust_layer/  
├── main.py  
├── config/  
│   └── config.yaml  
├── src/  
│   ├── guardrails.py  
│   └── server.py  
├── tests/  
│   └── test_guardrails.py  
└── pyproject.toml
```

Action Gateway

Action Gateway — Port 9999

Goal: Execute all external API calls on behalf of the LLM. The LLM only expresses intent via tool definitions — it never calls APIs directly.

What to build

src/mock_gateway.py — MockActionGateway

Implements ActionGatewayBase. Reads connector config at startup, builds tool definitions, routes tool calls to the appropriate HTTP connector endpoint.

Method	Behaviour
list_available_tools()	Return Anthropic-formatted tool definition list (read from config, cached)
execute(tool_call, session_id)	Route the tool call to the matching connector endpoint; return normalised result

Tool call input shape: {tool_name, tool_use_id, input_params}

Tool result shape: {tool_use_id, tool_name, result: dict, success: bool, error: str|None}

Only connector wired in PoC: onest_market_lookup

Tool definition:

name: onest_market_lookup

description: Search ONEST live job market data by trade and location

inputs:

- trade (required): electrician / welder / fitter
- location (optional): city name
- distance_km (optional): search radius, default 50

Config keys read:

- action_gateway.connectors.onest_market_lookup.endpoint — where to POST the lookup
- action_gateway.connectors.onest_market_lookup.timeout_ms — request timeout

Error handling: unknown tool → structured error dict; timeout → error: "onest_lookup_timeout";

HTTP error → error: "onest_http_error: {status_code}"

src/mock_server.py — Mock ONEST server (same port 9999)

A second FastAPI app that pretends to be the external ONEST API. Returns hardcoded fixture data by trade:

Trade	Salary Range	Signal
electrician	₹15k–₹28k	12% QoQ
welder	₹13k–₹22k	8% QoQ
fitter	₹14k–₹24k	10% QoQ
(any other)	₹12k–₹20k	stable

Endpoint: POST /onest/market_lookup → market data dict

Folder structure

```
action_gateway/  
├── main.py  
├── config/  
│   └── config.yaml  
├── src/  
│   ├── mock_gateway.py  
│   └── mock_server.py  
├── tests/  
│   ├── test_mock_gateway.py  
│   └── test_mock_server.py  
└── pyproject.toml
```

Reach Layer

Reach Layer — CLI client (no HTTP server)

Goal: Accept user input from a channel, deliver agent responses back. For the PoC, the channel is stdin/stdout.

What to build

src/cli_reach.py — CLIReachLayer

Implements ReachLayerBase. Wraps stdin/stdout as a channel.

Method	Behaviour
receive()	Block on input(), wrap the text in a TurnInput dataclass with a timestamp, raise EOFError on Ctrl-D
deliver(result)	Print Agent: {response_text} to stdout; add [Escalated to human agent] note if was_escalated=True
session_id (property)	Return the UUID generated at init (persists across the CLI session)

Config keys read: reach_layer.cli.prompt (input prefix), reach_layer.cli.agent_prefix (output prefix)

main.py — CLI REPL

Thin HTTP client. Not a server — runs as a foreground process.

Loop:

1. Call CLIReachLayer.receive() → get user message
2. POST {session_id, user_message, channel: "cli"} to http://localhost:8000/process_turn
3. Call CLIReachLayer.deliver(result) → print response
4. Repeat until EOFError or KeyboardInterrupt → exit cleanly

Error messages shown to user:

- Timeout → [Error: Agent Core did not respond in time. Is it running?]
- Connection refused → [Error: Could not connect to Agent Core at {endpoint}. Is it running?]
- HTTP error → [Error: Agent Core returned HTTP {status_code}]

Config keys read: agent_core_client.endpoint, agent_core_client.timeout_s

Folder structure

```
reach_layer/
├── main.py
├── config/
│   └── config.yaml
├── src/
│   └── cli_reach.py
├── tests/
│   └── test_cli_reach.py
└── pyproject.toml
```


Learning Layer

Learning Layer — Port 8004

Goal: Capture observability data on every turn. Runs fully out-of-band — never in the response path.

What to build

src/console_logger.py — ConsoleLogger

Implements LearningLayerBase. In the PoC, "logging" means writing structured entries via Python's logging module. In production this would write to a time-series store, vector DB for evals, and a feedback pipeline.

Method	Behaviour
emit_turn(event)	Log all turn fields: session, response, tools used, trust results, model, token counts, latency, timestamp. Accepts both dataclass and dict (duck-typed via getattr/.get)
emit_signal(signal_type, data)	Log a discrete signal event: drop_off, low_confidence, escalation_triggered, etc.

Both methods never raise — all exceptions are caught and logged as warnings internally.

Config keys read: learning_layer.log_level (default INFO)

src/server.py — FastAPI app

Endpoint	Method	Purpose
POST /emit/turn	Full TurnEventRequest body → {status: ok}	Receives async turn event from Agent Core after response is delivered
POST /emit/signal	{signal_type, data} → {status: ok}	Receives discrete signal events
GET /health	→ {status: ok}	Liveness probe

Turn event request carries: session_id, response_text, tool_calls, trust_input_result, trust_output_result, model_used, input_tokens, output_tokens, latency_ms, timestamp_ms

Folder structure

```
learning_layer/
├── main.py
├── config/
│   └── config.yaml
├── src/
│   ├── console_logger.py
│   └── server.py
├── tests/
│   └── test_console_logger.py
└── pyproject.toml
```

Domain Configuration Kit

Domain Configuration Kit

What Is It

The Domain Configuration Kit is the only thing that changes between deployments of the DPG Composition Framework. The 7 DPG blocks (Agent Core, Knowledge Engine, Trust Layer, Memory Layer, Action Gateway, Reach Layer, Learning Layer) are fixed, reusable code. The kit is what makes those blocks behave as a labour advisory bot for KKB, or a crop insurance advisor, or a healthcare helpline — without touching any Python code.

The core principle: zero domain-specific values in Python source. Every value that varies between deployments — persona text, intents, blocked phrases, knowledge documents, LLM model selection, connector endpoints — lives in YAML.

The Three-Layer Config Model

There are three distinct layers. Understanding the separation is critical.

Layer 1 — Per-Service Local Dev Config (each service's own config/config.yaml, localhost URLs, never shipped to production)

Each of the 7 services has its own {service}/config/config.yaml. This is the developer's local default — used when running services directly on a machine (not in Docker or Kubernetes). It uses localhost URLs and is never shipped to production. These configs are loaded once at service startup and never re-read during a request.

Service	Config Path	What It Controls
Agent Core	agent_core/config/config.yaml	LLM models, timeouts, retry logic, NLU settings, connector definitions
Knowledge Engine	knowledge_engine/config/config.yaml	Vector store, embedding model, RAG sources, glossary, persona, language instruction
Trust Layer	trust_layer/config/config.yaml	Blocked phrases (input + output), escalation topics
Memory Layer	memory_layer/config/config.yaml	Session TTL, max sessions
Action Gateway	action_gateway/config/config.yaml	Connector endpoints and timeouts
Reach Layer	reach_layer/config/config.yaml	Channel type (CLI/WhatsApp), agent_core endpoint

Learning Layer	learning_layer/config/ config.yaml	Log level, audit settings
----------------	---------------------------------------	---------------------------

These configs are loaded once at service startup. They are never re-read during a request.

Layer 2 — DPG Framework Defaults (deployment/dev-kit/dpg/)

Framework-level operational settings that are the same for any domain deployment. Defines how the DPG blocks behave technically — independent of what domain they serve.

What lives here: server ports, retry counts, timeout values, embedding provider and model, vector store type, session TTL defaults, log level defaults.

Layer 3 — Domain Configuration (deployment/dev-kit/configs/{domain}/)

Domain-specific values that make a deployment unique to a use case. This is the actual "kit" that a new deployer fills in.

What lives here: persona text, intents, glossary mappings, knowledge document sources, trust rules, LLM model selection, connector definitions, language settings, evaluation metrics.

What Goes in the Kit	What Does NOT Go in the Kit
Persona text, tone, name	Port numbers
Intents and entity types	Retry counts, timeout_ms
Glossary mappings	Server host/IP
Knowledge document sources	Embedding provider selection
Blocked phrases and escalation rules	Vector store type
LLM model selection	Infrastructure credentials
Connector definitions and tool names	Docker networking config
Language and locale settings	
Evaluation metrics	

What We're Building

Key insight from the codebase: Every service's main.py already has `_load_config(path: str = "config/config.yaml")` — the path is a parameter. For deployment, we volume-mount the deployment config into the container at `/app/config/config.yaml`. No code changes needed in any service.

Folder Structure

Configuration Kit

```
dev-kit/
├── __init__.py
├── schema.py
├── loader.py
├── dpg/
│   ├── agent_core.yaml
│   ├── knowledge_engine.yaml
│   ├── trust_layer.yaml
│   ├── memory_layer.yaml
│   ├── learning_layer.yaml
│   ├── action_gateway.yaml
│   └── reach_layer.yaml
└── configs/
    └── kkb/
        ├── agent_core.yaml
        ├── knowledge_engine.yaml
        ├── trust_layer.yaml
        ├── memory_layer.yaml
        ├── learning_layer.yaml
        ├── action_gateway.yaml
        └── reach_layer.yaml
```

Pydantic models -- validates DPG + domain configs
loads dpg/ + configs/{domain}/, merges, validates

DPG framework defaults -- never domain-specific
server port, retry_attempts, timeout_ms
vector_store, embedding_provider, top_k
server port only
session_ttl, max_sessions
server port, log_level
server port, timeout_ms
server port, **default** timeout

KKB domain values -- one file per **service**
primary_model, connectors, intents, endpoints
persona, glossary, sources, intent_filters
blocked_phrases, escalation_topics
TTL override **if** needed
log_level override **if** needed
onest_market_lookup endpoint
agent_core endpoint

Automation

```
automation/
├── docker/
│   ├── docker-compose.yml
│   └── .env.example
├── helm/
│   ├── agent-core/
│   │   ├── Chart.yaml
│   │   ├── values.yaml
│   │   └── templates/
│   │       ├── configmap-dpg.yaml          # from dev-kit/dpg/agent_core.yaml
│   │       ├── configmap-domain.yaml      # from dev-kit/configs/kkb/agent_core.yaml
│   │       ├── secret.yaml                # ANTHROPIC_API_KEY (only agent-core)
│   │       ├── deployment.yaml
│   │       └── service.yaml                # ClusterIP :8000
│   ├── knowledge-engine/
│   │   ├── Chart.yaml
│   │   ├── values.yaml
│   │   └── templates/
│   │       ├── configmap-dpg.yaml
│   │       ├── configmap-domain.yaml
│   │       ├── pvc.yaml                   # ChromaDB -- only knowledge-engine
│   │       ├── deployment.yaml
│   │       └── service.yaml               # ClusterIP :8001
│   ├── trust-layer/
│   │   ├── Chart.yaml
│   │   ├── values.yaml
│   │   └── templates/
│   │       ├── configmap-dpg.yaml
│   │       ├── configmap-domain.yaml
│   │       ├── deployment.yaml
│   │       └── service.yaml               # ClusterIP :8003
│   └── memory-layer/
│       ├── Chart.yaml
│       ├── values.yaml
│       └── templates/
│           └── configmap-dpg.yaml
```



```

├── configmap-domain.yaml
├── deployment.yaml
├── service.yaml # ClusterIP :8002
├── action-gateway/
│   ├── Chart.yaml
│   ├── values.yaml
│   └── templates/
│       ├── configmap-dpg.yaml
│       ├── configmap-domain.yaml
│       ├── deployment.yaml
│       └── service.yaml # ClusterIP :9999
├── learning-layer/
│   ├── Chart.yaml
│   ├── values.yaml
│   └── templates/
│       ├── configmap-dpg.yaml
│       ├── configmap-domain.yaml
│       ├── deployment.yaml
│       └── service.yaml # ClusterIP :8004
└── reach-layer/
    ├── Chart.yaml
    ├── values.yaml
    └── templates/
        ├── configmap-dpg.yaml
        ├── configmap-domain.yaml
        ├── deployment.yaml
        └── service.yaml # ClusterIP :8005

```

How the Config Flow Works

Local dev:

{service}/config/config.yaml (localhost URLs)
 → service reads its own config/config.yaml at startup

Docker:

dev-kit/configs/kkb/agent_core.yaml
 → mounted as volume into container at /app/config/config.yaml
 → service reads /app/config/config.yaml at startup (same code path, different file)
 → URLs use Docker service names (knowledge_engine:8001, not localhost:8001)

Helm / Kubernetes:

- dev-kit/configs/kkb/agent_core.yaml
- loaded into ConfigMap at helm install
- mounted as /app/config/config.yaml in each pod
- same result as Docker

The {service}/config/config.yaml files stay untouched — they remain the local dev defaults.

Two Folders, Two Purposes

dev-kit/ — The Domain Configuration Kit

What it is: The actual Domain Configuration Kit. This is the thing you hand to a new deployer when they want to run the framework for a new use case.

What it contains:

- **schema.py** — Pydantic models defining exactly what fields are valid, what types they must be, and what is required. Wrong value or missing field is caught before anything runs.
- **loader.py** — loads DPG defaults and domain YAMLs, merges each pair, validates against schema, returns structured config objects.
- **dpg/** — 7 YAML files with DPG framework defaults. Same for any domain. Never modified by a domain deployer.
- **configs/kkb/** — 7 YAML files with KKB-specific values: LLM model, persona, intents, trust rules, knowledge documents, connectors.

Why it exists: This is the only thing that changes between a labour advisory bot and a crop insurance bot. A new deployer creates **configs/agriculture/** — no Python changes, no Dockerfile changes, nothing else.

automation/docker/ — Run All 7 Services on One Machine

What it contains:

- **docker-compose.yml** — defines all 7 containers, startup order, healthchecks, networking, volume mounts, and the ChromaDB named volume for persistence.
- **.env.example** — template for ANTHROPIC_API_KEY. Deployer copies to .env, fills in the key.

Why it exists: Local development and demos. A single **docker compose up** starts everything.

Who uses it: Developers, demo environments, anyone who wants to run the full stack on one machine

automation/helm/ — Deploy to Kubernetes for Production

What it contains: One independent Helm chart per DPG block (7 charts total).

File	Purpose	Present in
configmap-dpg.yaml	DPG framework defaults for that block	All 7 charts
configmap-domain.yaml	Domain values for that block	All 7 charts
deployment.yaml	Mounts configs, sets resources (100m CPU / 512Mi), injects secrets	All 7 charts
service.yaml	ClusterIP for inter-service communication	All 7 charts
pvc.yaml	ChromaDB persistent storage	knowledge-engine only
secret.yaml	ANTHROPIC_API_KEY	agent-core only

Why it exists: Production deployments. Each block deploys and upgrades independently:

```
helm install agent-core automation/helm/agent-core/ --namespace agent-core
helm install knowledge-engine automation/helm/knowledge-engine/ --namespace knowledge-engine
```

Upgrading only Knowledge Engine (e.g. new glossary mappings):

```
helm upgrade knowledge-engine automation/helm/knowledge-engine/ --namespace knowledge-engine
```

Other 6 services are not touched.

Who uses it: DevOps / platform teams deploying to cloud infrastructure.

How dev-kit/ flows into docker/ and helm/

The connection is volume mounts. The domain config YAMLs are injected into each running container at the path the service expects: /app/config/config.yaml.

Docker Compose — you reference the domain config files directly in docker-compose.yml:

```
# automation/docker/docker-compose.yml
services:
  agent_core:
    build: ../../../../agent_core
```

```

volumes:
  - ../../dev-kit/configs/kkb/agent_core.yaml:/app/config/config.yaml

knowledge_engine:
  build: ../../../../knowledge_engine
  volumes:
    -
      ../../dev-kit/configs/kkb/knowledge_engine.yaml:/app/config/config.yaml

```

The container starts, the service code runs `_load_config("config/config.yaml")`, and it reads the KKB domain config — not the local dev default.

Helm — the domain config YAML content is embedded into a ConfigMap, then mounted:

```

# configmap-domain.yaml
data:
  config.yaml: |
    {{ .Files.Get "../../dev-kit/configs/kkb/agent_core.yaml" | indent 4
}}

# deployment.yaml
volumeMounts:
  - name: domain-config
    mountPath: /app/config/config.yaml
    subPath: config.yaml

```

When someone changes domain_config for their domain:

- **For Docker:** docker compose up — mounts whatever is in dev-kit/configs/their_domain/
- **For Helm:** helm upgrade {block} automation/helm/{block}/ — Kubernetes picks up new ConfigMaps
- **No rebuild of Docker images needed — the images are generic. Only the mounted config changes.**

DPG Services — Minimum Resource Requirements

DPG Services — Minimum Resource Requirements

What drives resource usage per service

Service	Port	Type	What's heavy
memory_layer	8002	Stub	Nothing heavy — pure Python dict in memory
trust_layer	8003	Stub	Nothing heavy — string matching against blocked phrases
learning_layer	8004	Stub	Nothing heavy — console log writes only
action_gateway	9999	Stub	Nothing heavy — returns hardcoded JSON
knowledge_engine	8001	Full	sentence-transformers model (~470 MB on disk) + ChromaDB in-process vector store
agent_core	8000	Full	Nothing local — all LLM work is remote (Anthropic API)
reach_layer	CLI	Stub	Nothing heavy — stdin/stdout + one HTTP client

Minimum resource table

Service	Min RAM	Min CPU	Notes
memory_layer	128 MB	0.05 CPU	Python process baseline ~50 MB + session state
trust_layer	128 MB	0.05 CPU	Python process baseline ~50 MB + rule config
learning_layer	128 MB	0.05 CPU	Python process baseline ~50 MB + log buffer
action_gateway	128 MB	0.05 CPU	Python process baseline ~50 MB + fixture data
knowledge_engine	1.5 GB	0.5 CPU	Model load: ~470 MB · ChromaDB: ~200 MB · Python: ~100 MB · ingest CPU spike
agent_core	256 MB	0.1 CPU	Python + httpx clients to 5 services + request/response buffers
reach_layer	64 MB	0.05 CPU	Barely anything — just httpx + stdin
Total	~2.3 GB	~0.85 CPU	All services running simultaneously

Recommended limits for local Docker (what's in the compose files now)

Service	RAM limit	CPU limit
memory_layer	512 MB	0.1
trust_layer	512 MB	0.1
learning_layer	512 MB	0.1
action_gateway	512 MB	0.1
knowledge_engine	2 GB	0.5
agent_core	512 MB	0.1
reach_layer	512 MB	0.1

Key facts for the record

- Knowledge Engine is the only service that can OOM. The paraphrase-multilingual-MiniLM-L12-v2 model needs ~470 MB just to load; ingest spikes above 1 GB — setting it below 1.5 GB risks an OOM kill during startup. To reduce this to ~256 MB, switch embedding_provider: openai in dev-kit/dpg/knowledge_engine.yaml and set OPENAI_API_KEY — embeddings are then generated via API with no local model loaded.
- Agent Core has zero local ML. All LLM work goes to Anthropic API over HTTPS. Its memory is just Python + 5 httpx client connections.
- The 4 stubs (Memory, Trust, Learning, Action) are interchangeable at 128 MB minimum — they only become heavier when replaced with real implementations (e.g., a real Trust Layer with an ML safety model could need 2–4 GB).
- Sentence-transformers runs on CPU. Embedding at query time adds ~50–150 ms per turn on a modern laptop CPU.

KKB Profile Building Flow — Complete Design

KKB Profile Building Flow — Complete Design

1. Overview

This design covers the end-to-end flow from a new user's first message through to being ready for job search and apply. It replaces the current hardcoded stub profile with a Redis-backed persistent store and introduces a workflow_step state machine that drives all routing decisions.

Scope of this design:

- New user detection and consent flow
- Profile collection (consent=true path)
- Anonymous session (consent=false path)
- Returning user session resume
- Profile conflict detection
- Redis abstraction layer
- NLU conditional execution
- Hard minimum fields and grace turn
- Session end cleanup

2. Entities Collected — 5-Layer Entity Map

From sample_usecase.pdf. These are the fields the system knows about a user.

Layer	Fields	How collected
Identity	name, age, gender, disability_status, location	Asked in Round 1 + 2
Capability	trade_or_stream, iti_pass_year, iti_institute, certifications	Asked in Round 2 + 3
Constraint	max_distance_km, preferred_shift, salary_expectation_min	Asked in Round 3
Aspiration	target_roles, preferred_sectors, open_to_relocation	Asked in Round 4
Journey	application_status, jobs_viewed, last_interaction, sessions_count	System-tracked only, never asked

Hard minimum fields (required for ONEST query):

- capability.trade_or_stream
- identity.location

These fields are collected as part of the normal 5-round flow (trade in Round 3, location in Round 2). After all 5 rounds complete, the system checks whether both are present in the profile.

- **If both are already collected** — no additional ask. Proceed directly to workflow_step = "ready".
- **If either is missing** (user skipped that round) — one grace turn asks only the missing field(s). Whether the user answers or skips the grace turn, the system proceeds to ready regardless.

Inferred fields (never asked, written by system):

- **language_preference** — from Language Normaliser detected_language at session start

3. Workflow Step State Machine

Three states only. Consent differences (TTL, cleanup) are handled via profile.consent flag — no separate states needed. In future, if rounds or tools need to differ by consent, check profile.consent at runtime.

New user:

- no profile → awaiting_consent → agrees → profile_building (5 rounds) → grace turn if needed → ready
- no profile → awaiting_consent → declines → profile_building (5 rounds, session-only) → grace turn if needed → ready → session ends → DELETE all

Returning user:

- profile exists, consent=true, incomplete session → offer resume → ready (resumed or fresh)
- profile exists, consent=true, complete profile → ready

workflow_step	Meaning
awaiting_consent	New user, consent not yet answered
profile_building	Collecting profile fields (both consent paths)
ready	Profile complete, full service available

4. Two Parallel State Machines

workflow_step only governs profile setup. Once **ready**, a second state machine takes over:

State machine	Field	Values	Purpose
Profile state	workflow_step	awaiting_consent → profile_building → ready	Governs first-time setup only

Journey state	current_mental_state	fog → orientation → evaluation → commitment → follow-through	Governs conversation flow after ready
---------------	----------------------	--	---------------------------------------

When a returning user connects, the system reads **current_mental_state** from their profile and resumes from that point — not from fog. A user who was at commitment (decided to apply) is asked "did you hear back from that employer?" not "kya kaam dhundh rahe ho?"

5. Storage Backend — Abstract Interface

All Redis interaction is behind a common interface (MemoryBackendBase). No other module imports Redis directly. Swapping to a different datastore only requires a new implementation class — no changes to any other module.

6. Redis Schema

Three key types, three distinct purposes.

- **profile:{phone_number}** RedisJSON Permanent (consent=true) / TTL 4h (consent=false)
- **session:{session_id}** RedisJSON TTL 24h (consent=true) / TTL 4h (consent=false)
- **user_sessions:{phone_number}** Sorted Set Permanent (entry deleted at session end for consent=false)

profile:{phone_number} — RedisJSON

```
{
  "phone_number": "+919876543210",
  "consent": true,
  "language_preference": "hindi",
  "identity": {
    "name": "Ravi",
    "age": 22,
    "gender": "male",
    "disability_status": null,
    "location": "Hubli"
  },
  "capability": {
    "trade_or_stream": "electrician",
    "iti_pass_year": 2023,
    "iti_institute": null,
    "certifications": []
  }
}
```

```

    },
    "constraint": {
      "max_distance_km": 50,
      "preferred_shift": null,
      "salary_expectation_min": null
    },
    "aspiration": {
      "target_roles": [],
      "preferred_sectors": [],
      "open_to_relocation": false
    },
    "journey": {
      "application_status": {},
      "jobs_viewed": [],
      "last_interaction": 1711276800,
      "sessions_count": 3
    }
  }
}

```

Why RedisJSON: Atomic field-level updates without read-modify-write. Example: `JSON.SET profile:+91xxx $.identity.location "Dharwad"` updates only location without fetching the whole document.

session:{session_id} — RedisJSON

```

{
  "session_id": "sess_abc123",
  "phone_number": "+919876543210",
  "workflow_step": "profile_building",
  "collection_round": 2,
  "pending_profile_update": null,
  "history": [
    {
      "turn": 1,
      "user": "naukri chahiye",
      "assistant": "Zaroor! Pehle thodi jaankari...",
      "tool_calls": [],
      "timestamp": 1711276800
    }
  ],
  "started_at": 1711276800,
  "last_active": 1711276860
}

```

Why RedisJSON here too: workflow_step, collection_round, and confirmed_entities are updated independently on almost every turn. Atomic path-level updates avoid race conditions and eliminate full-document serialization on each write.

collection_round — tracks which of the 5 rounds the system is currently on. Without this, after a session resume the system would not know where it left off and would restart collection from Round 1.

user_sessions:{phone_number} — Sorted Set

ZADD user_sessions:+919876543210 1711190400 "sess_xyz789" ← older session

ZADD user_sessions:+919876543210 1711276800 "sess_abc123" ← most recent

Why a separate key: Sessions are keyed by session_id. When a returning user connects, you know their phone_number but not their session_id. This sorted set is the reverse index — it maps phone number → list of session IDs ordered by time. Without it, you cannot find the user's session history from just their phone number.

Two questions it answers:

- "Does this user have any previous sessions?" → ZCARD user_sessions:{phone_number}
- "What is their most recent session?" → ZREVRANGE user_sessions:{phone_number} 0 0

NOTE: Profile keys and session keys use separate Redis logical databases (configured via redis.profile_db and redis.session_db in domain.yaml). This keeps profile data and session data isolated — separate keyspace, separate eviction policies, and easier to swap one store independently of the other in future.

7. Turn Start Sequence (Step 1 — Memory Read)

```
Step 1a: get_profile(phone_number)
        → empty dict = new user
        → profile with consent=false = was declined (treat as new if TTL expired, else ready)
        → profile with consent=true = returning user

Step 1b: (only if profile exists and consent=true)
        get_user_sessions(phone_number, limit=1)
        → empty list = no prior sessions (first turn after profile was built)
        → [session_id] = found, load it
```

```
Step 1c: (only if session_id found)
    get_session(session_id)
    → check workflow_step, collection_round, history
```

8. New User Flow — Consent=True Path

Turn 1 — Consent ask

System detects: get_profile(phone_number) returns empty dict.

Creates **session:{session_id}** with **workflow_step = "awaiting_consent"**.

Manager Agent selects the consent system prompt. LLM asks:

```
"Namaste! Main KKB hoon -- kaam dhundhne mein madad karta hoon. Behtar
madad ke liye aapki thodi jaankari store karna chahta hoon. Kya aap consent
dete hain?"
```

Turn 2 — User agrees

Trust Layer detects consent signal (configured phrases in domain.yaml).

Actions:

- set_profile(phone_number, {consent: true, ...empty layers...})
- patch_session(session_id, "workflow_step", "profile_building")
- Write language_preference to profile from Language Normaliser output (inferred, never asked)
- patch_session(session_id, "collection_round", 0)

LLM immediately starts Round 1.

Rounds 1–5 — Collection

Manager Agent selects profile_building system prompt. NLU runs in entity-extraction-only mode (no intent classification needed — workflow_step already determines routing).

Round	Fields asked	Layer(s)
1	name, age, gender	Identity
2	disability_status, location	Identity
3	trade_or_stream, iti_pass_year, iti_institute	Capability
4	max_distance_km, salary_expectation_min, preferred_shift	Constraint
5	target_roles, open_to_relocation	Aspiration

Each round: LLM asks 2–3 related fields together. User answers (fully, partially, or skips). Entity extractor pulls values from the response. Confirmed entities written profile:{phone_number} via patch_profile. collection_round incremented.

After Round 5 — Hard minimum check

```
hard_minimum_met = (  
    profile.get("capability", {}).get("trade_or_stream") and  
    profile.get("identity", {}).get("location")  
)
```

```
if not hard_minimum_met:  
    # Grace turn -- one final ask, both fields together  
    ask: "Behtar job results ke liye -- aap kaunsa kaam karte hain aur  
    kahan se hain?"  
    # Wait one turn. Whether user answers or not, proceed.  
    workflow_step = "ready"  
else:  
    workflow_step = "ready"
```

One grace turn maximum. If user skips it, system proceeds without those fields. ONEST tool handles missing parameters gracefully.

workflow_step = "ready"

Full service available. Manager Agent selects full NLU + full tool set. KE RAG available as a tool the LLM can call. ONEST search available.

9. New User Flow — Consent=False Path

For the consent=false path, the profile collection flow is identical to the consent=true path — all 5 rounds are completed with the system informing the user that their details are for this session only — the only difference is that the profile and session data are permanently deleted at session end.

workflow_step = "ready"

Full service available for this session. Journey layer fields written to session only.

Session end — Cleanup

Triggered by: exit phrase detected by Trust Layer, session TTL expiry, or Reach Layer session_close event.

```
backend.delete_session_and_profile(phone_number, session_id)  
# Deletes: profile:{phone_number}, session:{session_id}  
# Removes: session_id from user_sessions:{phone_number}  
# If user_sessions is now empty, delete that key too
```

9. Returning User Flow

Case A — Profile complete, active session:

Load last 3 turns (configurable) from history. workflow_step = "ready". Resume at current_mental_state.

Case B — Profile incomplete, session interrupted:

collection_round < total_rounds. Offer resume in first response. User agrees → restore collection_round, continue. User declines → reset collection_round = 0, restart.

10. Profile Conflict Detection

Applies to fields listed in profile_collection.conflict_sensitive_fields in domain.yaml. Default: identity.location and capability.trade_or_stream — these directly affect ONEST query results.

- If a new value differs from stored value for a sensitive field → store as pending_profile_update in session, ask user to confirm before writing
- User confirms → patch_profile, clear pending_profile_update
- User declines → clear pending_profile_update, keep old value
- All other fields overwritten directly without confirmation

The sensitive fields list is configurable — a different domain can protect different fields.

11. NLU Conditional Execution

NLU is the most expensive pre-LLM step. Running it when the result will be ignored wastes latency.

workflow_step	NLU mode	Reason
awaiting_consent	Skip entirely	Only listening for consent signals – detected by Trust Layer phrase match
profile_building	Entity extraction only	Routing fixed by workflow_step, extract field values only
ready	Full NLU	Intent + entities + sentiment

Manager Agent checks **workflow_step** before invoking Knowledge Engine and passes the appropriate NLU mode flag.

Consent phrase detection: Trust Layer phrase list first, LLM fallback if no match. Phrase list is configurable in domain.yaml under trust.consent_phrases and trust.decline_phrases.

12. Manager Agent Routing

Manager Agent selects system prompt and tools before the LLM call based on **workflow_step**..

workflow_step	System prompt	Tools
awaiting_consent	consent_prompt	none
profile_building	profile_prompt	none
ready	main_prompt	knowledge_search, onest_search, onest_apply

All prompt blocks and tool lists come from **domain.yaml** — nothing hardcoded in Python.

13. Knowledge Engine — Conditional Call

Current behaviour (wrong): KE is called on every turn unconditionally.

Correct behaviour: KE RAG is a tool the LLM calls when it needs domain knowledge. It is not a mandatory pre-step.

- For awaiting_consent, profile_building: KE is not in the tools list → never called
- For ready: KE is in the tools list as knowledge_search → LLM calls it only when needed

This eliminates the RAG call on consent/profile turns (which have no need for domain documents).

14. Journey Layer — System Tracked

Journey fields are never asked. They are written by Agent Core after each turn, asynchronously after the response is delivered.

- **session_count** — incremented at the start of each new session
- **current_mental_state** — fog / orientation / evaluation / commitment / follow-through — updated by the system based on conversation signals
- **last_market_truth_delivered** — timestamp and summary of the last job market information delivered to the user
- **options_presented** — list of job options shown to the user, with timestamps
- **actions_taken** — applied / enrolled / counsellor_connected
- **outcomes_tracked** — callback received / joined / left / unknown
- **drop_off_reason** — recorded if applicable when session ends without resolution

For consent=false path, all journey fields are still written to profile during the session — deleted at session end.

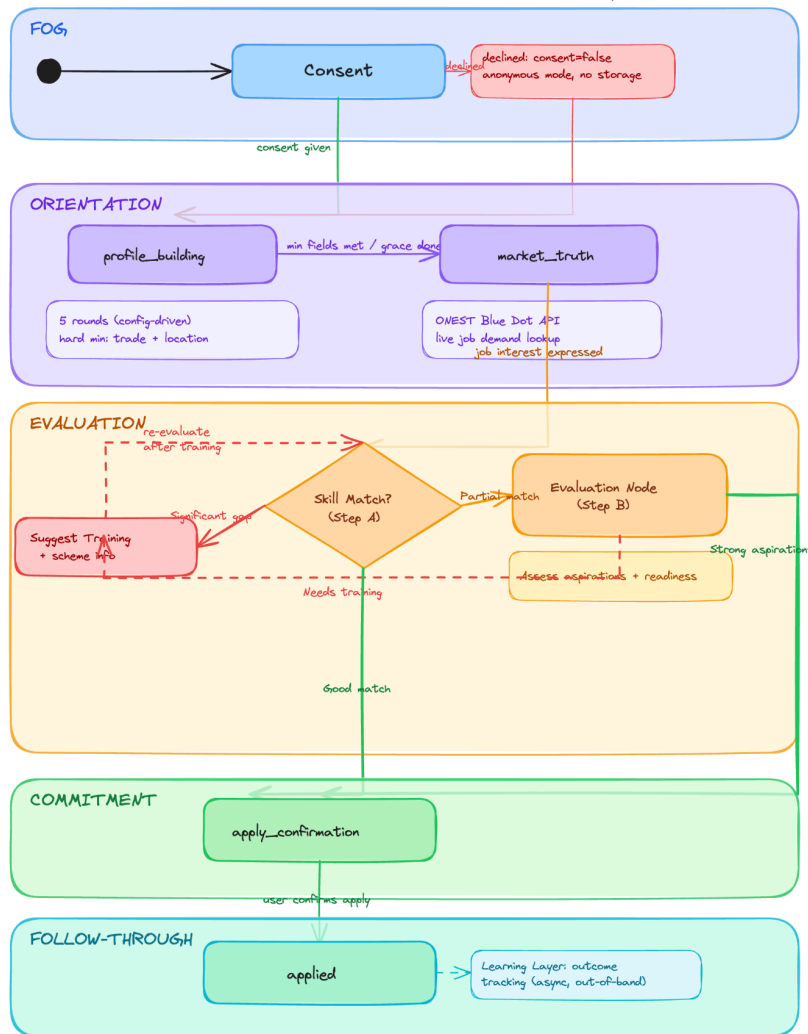
15. ONEST Apply Behaviour

For the current PoC, **onest_apply** applies with whatever profile data is available. No mandatory field check. The ONEST apply API returns **applied: true** for all requests. Mandatory field validation will be added when the real API is integrated.

State management flow

KKB — Hierarchical State Machine

outer box = mental_state | inner node = workflow_step



Returning User Entry Points

Full profile (hard min fields met)
=> resume at market_truth

Partial profile (some fields missing)
=> resume at profile_building

Active session (within TTL)
=> resume at last workflow_step

consent=false / no profile
=> restart at awaiting_consent

NLU mode by workflow_step:
awaiting_consent => skip NLU
profile_building => entities only
market_truth+ => Full NLU + RAG,
session_resume_turns: 3 (config)
conflict_sensitive_fields in domain.yaml

mental_state: FOG

State: awaiting_consent

Trigger: New user arrives (no profile in Redis)

User response	Branch	Next state
Agrees (consent phrases match)	Store profile.consent=true	profile_building → mental_state becomes ORIENTATION

Declines (decline phrases match)	Store profile.consent=false, TTL=4h	profile_building (session-only, no persistence)
No clear response	Re-prompt (same state)	awaiting_consent

mental_state: ORIENTATION

State: profile_building

Trigger: Consent resolved (either path). System collects entities over 5 rounds (config-driven). After each round, extracted entities written to profile (if consent=true) or session-only (if consent=false).

Condition	Branch	Next state
collection_round >= total_rounds, both hard minimum fields present	Skip grace turn	market_truth
collection_round >= total_rounds, hard minimum missing	Ask grace turn (1 exchange)	market_truth (regardless of grace turn answer)
User provides conflicting value for conflict_sensitive_fields	Hold in pending_profile_update, ask to confirm	Stay in profile_building
User drops off mid-collection	Save collection_round to session, session expires	Resume from same round on return

NLU mode: entity extraction only (no intent classification, no RAG)

State: market_truth

Trigger: Profile collection complete (hard minimum met or grace turn done)

Condition	Branch	Next state
User expresses job search intent	Present live ONEST job demand data	skill_evaluation → mental_state becomes EVALUATION
User asks about schemes / training	Answer via RAG, stay in orientation	market_truth
User provides new profile info	Update profile entities	market_truth
Returning user (full profile, new session)	Skip profile_building, load profile	market_truth (entry point)

mental_state: EVALUATION

State: skill_evaluation — Step A (Skill Match Check)

Trigger: User ready for job search. System compares profile entities against ONEST demand data.

Condition	Branch	Next state
Strong match (trade + location align with demand)	Present matching jobs directly	apply_confirmation → mental_state becomes COMMITMENT
Partial match (trade matches, location/experience gap)	Deeper assessment needed	skill_evaluation Step B
Significant gap (trade mismatch or no demand)	Suggest relevant training + govt schemes	skill_evaluation Step A (loop – re-evaluate after training)
User explicitly says "not interested in jobs now"	Acknowledge, stay available	market_truth

State: skill_evaluation — Step B (Evaluation Node)

Trigger: Partial match detected in Step A

Condition	Branch	Next state
User shows strong aspiration + willingness to relocate / upskill	Treat as viable candidate	apply_confirmation → mental_state becomes COMMITMENT
User shows significant hesitation or constraints	Suggest targeted training path + schemes	skill_evaluation Step A (re-enter after training)
User drops off during evaluation	Save skill_evaluation step to session	Resume at Step B on return

mental_state: COMMITMENT

State: apply_confirmation

Trigger: Good match confirmed (via Step A or Step B)

User response	Branch	Next state
Confirms intent to apply	Log outcome, trigger Action Gateway (write connector – consent-gated)	applied → mental_state becomes FOLLOW-THROUGH

Wants more job options	Re-run ONEST query with adjusted params	skill_evaluation Step A
Changes mind / not ready	Acknowledge, preserve profile state	market_truth

Trust Layer: write connector requires explicit consent check before Action Gateway call

mental_state: FOLLOW-THROUGH

State: applied

Trigger: User confirmed application intent

Condition	Branch	Next state
User returns, asks about application status	Query Action Gateway (read connector)	applied (stay, provide update)
User wants to apply for another job	Re-run job match	skill_evaluation Step A
Session ends	Emit outcome event to Learning Layer (async)	—

Learning Layer: outcome tracked async after response. Never in response path.

Returning User Entry Points

Profile state on return	Entry point
No profile / expired	awaiting_consent (FOG)
consent=false profile	awaiting_consent (re-ask on new session)
Partial profile (hard min missing)	profile_building (resume at saved collection_round)
Full profile (hard min met)	market_truth (skip collection entirely)
Active session within TTL	Resume at exact workflow_step saved in session

Memory Layer — Context Graph Design

Memory Layer — Context Graph Design

Core Principle

The Memory Layer uses two stores with a clear separation of concerns:

- **Neo4j** holds the *context graph* — everything the LLM needs to respond correctly. Session identity, operational state (`workflow_step`, `collection_round`), the last question asked (`current_question`), and all collected profile knowledge as typed Attribute nodes. One Neo4j query per turn gives the LLM its complete context — no history needed.
- **SQLite** holds *audit and cross-session data* — consent records, session registry (phone → `session_id` lookup), turn history (audit log only, never read into LLM context), and cross-session profile snapshots for returning users.

Turn messages are sequential text data. They are written to SQLite for audit and compliance. They are **NOT** read back into the LLM context path — the context graph in Neo4j is sufficient. Neo4j is freed to do what only a graph store can: represent structured knowledge via typed relationship edges.

What is the Context Graph

The context graph represents what the system knows about the user in this session. Each piece of knowledge is an **Attribute node**. The relationship type connecting it to the session IS the semantic label — it is not a property, it is the edge itself.

```
(Session)
-[:HAS_TRADE]→      (Attribute {value: "electrician"})
-[:HAS_EDUCATION_LEVEL]→ (Attribute {value: "iti"})
-[:HAS_LOCATION]→    (Attribute {value: "hubli"})
-[:HAS_INCOME_URGENCY]→ (Attribute {value: "high"})
```

This is not a flat key-value map. It is a typed knowledge graph. The relationship type (HAS_TRADE) carries the meaning. The graph can be queried by relationship type, extended with new types without schema changes, and in future can model relationships between attributes themselves.

The relationship type names (HAS_TRADE, HAS_EDUCATION_LEVEL etc.) are domain-specific — they come from config, not from code. The Memory Layer code never hardcodes these strings.

Neo4j — Context Graph

Node: Session

The anchor of the context graph. One node per session. Holds session identity, operational state, and conversational state — workflow_step, collection_round, and current_question all live here, not in SQLite.

```
(Session {  
  session_id: "s1",  
  phone: "9876543210",  
  workflow_step: "profile_building",  
  collection_round: 1,  
  current_question: "aapka trade kya hai?"  
})
```

Node: Attribute

One node per collected profile field. Holds the value. The edge type gives it meaning.

```
(Attribute { value: "electrician" })  
(Attribute { value: "iti" })  
(Attribute { value: "hubli" })
```

Edges — Domain Profile (from config, not hardcoded)

Edge Type	Meaning	Example Value
HAS_LANGUAGE_PREFERENCE	User's detected language	hindi
HAS_EDUCATION_LEVEL	Highest qualification	iti
HAS_TRADE	Primary trade/skill	electrician
HAS_LOCATION	User's location	hubli
HAS_EXPERIENCE_YEARS	Work experience	2
HAS_COMMUTE_PREFERENCE	Max commute km	15
HAS_INCOME_URGENCY	How urgently income is needed	high
HAS_SALARY_EXPECTATION	Expected salary	15000
HAS_SECTOR_PREFERENCE	Preferred work sector	construction
HAS_TRAINING_PREFERENCE	Open to training	yes
HAS_GROWTH_HORIZON	Career growth horizon	short

These edge type names live in `memory_layer/config/domain.yaml` under `profile_collection.profile_graph_relations`. A different domain deployment replaces this config section entirely — the Memory Layer code is untouched.

Full Graph After Profile Collection (KKB Example)

```
(Session {session_id:"s1", phone:"9876543210", workflow_step:"market_truth",  
  collection_round:2, current_question:"aapke liye kaunsi jobs hain,  
  dekhein?"})
```

```
-[:HAS_LANGUAGE_PREFERENCE]→ (Attribute {value:"hindi"})
```

```
-[:HAS_EDUCATION_LEVEL]→ (Attribute {value:"iti"})
```

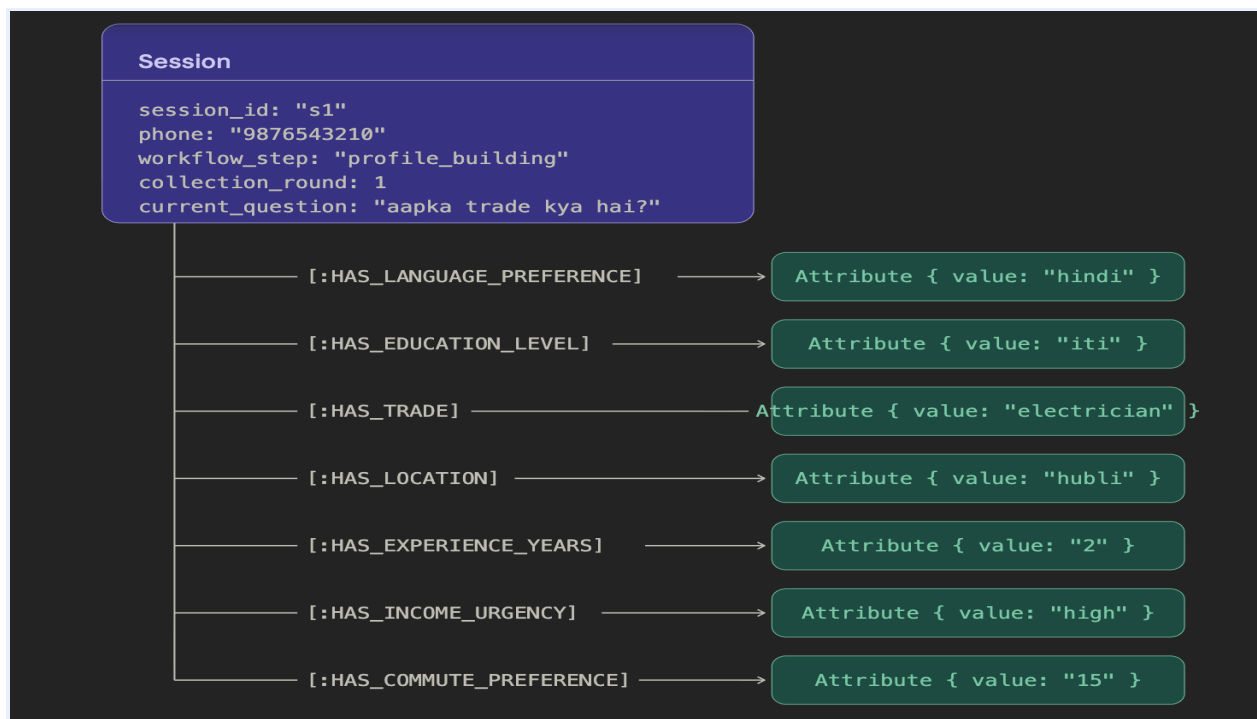
```
-[:HAS_TRADE]→ (Attribute {value:"electrician"})
```

```
-[:HAS_LOCATION]→ (Attribute {value:"hubli"})
```

```
-[:HAS_EXPERIENCE_YEARS]→ (Attribute {value:"2"})
```

```
-[:HAS_INCOME_URGENCY]→ (Attribute {value:"high"})
```

```
-[:HAS_COMMUTE_PREFERENCE]→ (Attribute {value:"15"})
```



That is the entire Neo4j graph per session. Clean. No turn history. No event nodes.

When Neo4j is Written

Trigger	Operation
New session starts	CREATE (s:Session {session_id, phone, workflow_step, collection_round, current_question: ""})
Profile field collected (async)	MERGE (s)-[:HAS_TRADE]->(a:Attribute) SET a.value = \$value — MERGE ensures update in place
Language detected for first time (async)	MERGE (s)-[:HAS_LANGUAGE_PREFERENCE]->(a:Attribute) SET a.value = \$lang
User corrects a field (async)	Same MERGE — overwrites the value
Every turn completes (async)	MATCH (s:Session {session_id: \$id}) SET s.workflow_step = ..., s.collection_round = ..., s.current_question = \$agent_response
Session ends with consent_granted=0	MATCH (s:Session) WHERE s.session_id = \$session_id DETACH DELETE s — removes graph for this session only
DPDP right-to-erasure request	MATCH (s:Session) WHERE s.session_id IN \$ids DETACH DELETE s — removes all sessions for this phone

All writes after the first session creation are **async** — they happen in a daemon thread after the response is delivered.

When Neo4j is Read

Trigger	Query
Every turn start (Step 1)	Load Session node properties + all Attribute nodes for this session_id — one query
Returning user — continue session	Same load, using session_id from SQLite lookup (phone → session_id)

The Load Query

```
MATCH (s:Session {session_id: $session_id})
OPTIONAL MATCH (s) - [r] -> (a:Attribute)
RETURN s.workflow_step AS workflow_step,
       s.collection_round AS collection_round,
       s.current_question AS current_question,
       s.phone AS phone,
       type(r) AS rel_type,
       a.value AS value
```

workflow_step	collection_round	current_question	rel_type	value
profile_building	1	"aapka trade kya hai?"	HAS_LANGUAGE_PREFERENCE	hindi
profile_building	1	"aapka trade kya hai?"	HAS_EDUCATION_LEVEL	iti

SQLite — Sequential and Relational Data

Schema

```

-- Consent and user identity
CREATE TABLE users (
    phone          TEXT PRIMARY KEY,
    consent_granted INTEGER NOT NULL DEFAULT 0,
    consent_at     INTEGER,
    created_at     INTEGER NOT NULL
);

-- Session registry
CREATE TABLE sessions (
    session_id     TEXT PRIMARY KEY,
    phone          TEXT NOT NULL,
    created_at     INTEGER NOT NULL,
    last_active_at INTEGER NOT NULL,
    FOREIGN KEY (phone) REFERENCES users(phone) ON DELETE CASCADE
);

-- Full conversation history (audit log only)
CREATE TABLE turns (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id      TEXT NOT NULL,
    turn_index      INTEGER NOT NULL,
    system_message  TEXT,
    user_message    TEXT,
    agent_response  TEXT,
    intent          TEXT,
    sentiment       TEXT,
    timestamp_ms    INTEGER NOT NULL,
    FOREIGN KEY (session_id) REFERENCES sessions(session_id) ON DELETE CASCADE
);

-- Profile snapshot
CREATE TABLE user_profile (
    phone          TEXT PRIMARY KEY,
    profile_json    TEXT NOT NULL,
    updated_at     INTEGER NOT NULL,
    FOREIGN KEY (phone) REFERENCES users(phone) ON DELETE CASCADE
);

CREATE INDEX idx_sessions_phone ON sessions(phone);

```

```
CREATE INDEX idx_turns_session_id ON turns(session_id, turn_index);
```

ON DELETE CASCADE — deleting from users where phone=? wipes everything for that user across all tables automatically.

When SQLite is Written

Trigger	Table	Operation	Sync/Async
Consent granted	users	INSERT	Sync
New session created	sessions	INSERT	Sync
Every turn completes	turns	INSERT (audit log only)	Async
Every turn completes	sessions	UPDATE last_active_at	Async
profile_building completes AND consent_granted=1	user_profile	INSERT OR REPLACE	Async
Session ends with consent_granted=0	user_profile	DELETE WHERE phone=? (targeted delete — sessions and turns are kept as audit)	Async

When SQLite is Read

Trigger	Table	Query	Purpose
Session start	sessions JOIN users	SELECT s.session_id, s.last_active_at, u.consent_granted FROM sessions s JOIN users u ON s.phone=u.phone WHERE s.phone=? ORDER BY s.last_active_at DESC LIMIT 1	Get session_id + consent_granted in one query. If no row → new user.
Returning user — fresh start	user_profile	SELECT profile_json WHERE phone=?	Pre-populate new Neo4j graph with known profile
DPDP erasure — get Neo4j IDs first	sessions	SELECT session_id WHERE phone=?	Get all session IDs to pass to Neo4j for DETACH DELETE
DPDP erasure — full wipe	users	DELETE WHERE phone=?	CASCADE deletes sessions, turns, user_profile automatically

SessionContextGraph — What Agent Core Works With

SessionContextGraph:

```

session_id      ← from Neo4j Session node
phone           ← from Neo4j Session node
workflow_step   ← from Neo4j Session node

```

collection_round ← from Neo4j Session node
current_question ← from Neo4j Session node (last question asked by agent)
language_preference ← from Neo4j (HAS_LANGUAGE_PREFERENCE Attribute)
profile_fields ← from Neo4j (all Attribute nodes, keyed by field name)
confirmed_entities ← derived from profile_fields (subset from NLU entity extraction)
consent_granted ← from SQLite users table (read at session start alongside session lookup)

Sources at load time:

SQLite (session start only — one query):

sessions JOIN users → session_id, last_active_at, consent_granted

Neo4j (every turn — one query):

Session node → session_id, phone, workflow_step, collection_round, current_question

Attribute nodes → profile_fields dict, language_preference

SQLite turns: NOT read at any point. Audit-only write log.

One SQLite query (session start) + one Neo4j query (every turn) = full SessionContextGraph.

How Agent Core Uses the Context Graph

Step	Field Used	How
Step 4 — NLU	current_question + workflow_step	Context-aware intent classification. Knows what the user is responding to — replaces history.
Step 5 — Workflow routing	workflow_step	Route to correct handler

Step	Field Used	How
Step 5 — Profile logic	collection_round + profile_fields	Check missing fields; determine what to ask next
Step 6 — KE retrieve	confirmed_entities + NLU entities	Merged entity set passed to KE.retrieve() as source filter for RAG
Step 7 — Manager Agent	profile_fields + workflow_step + current_question	Injected into system prompt — LLM knows full user context
Step 7 — Manager Agent	language_preference	System prompt instructs LLM to respond in user's language
Step 7 — Manager Agent	(current user message only)	build_messages() sends only the current turn — no prior history

How the Graph Is Updated After a Turn

Orchestrator builds updated state before launching async thread. Async thread calls `write_session()` which does:

- **Neo4j updates** (single MATCH, multiple SETs):
 - SET s.workflow_step = \$next_workflow_step
 - SET s.collection_round = \$next_collection_round
 - SET s.current_question = \$agent_response — response just delivered becomes next turn's context
 - For every new profile field collected → MERGE the corresponding Attribute node

- For language detected → MERGE the HAS_LANGUAGE_PREFERENCE Attribute
- **SQLite updates** (audit only):
 - INSERT INTO turns — full turn record
 - UPDATE sessions SET last_active_at = \$now
 - If profile_building just completed → INSERT OR REPLACE INTO user_profile

Returning User Flow

Session start: phone = "9876543210"

SQLite READ → SELECT session_id, last_active_at FROM sessions WHERE phone=? ORDER BY last_active_at DESC LIMIT 1

→ row found: {session_id:"s1", last_active_at:1704067200000}

→ returning user – agent offers to continue or start fresh

Path A -- Continue session:

Neo4j READ → load Session node (workflow_step, collection_round, current_question) + Attribute nodes (profile_fields)

→ resume exactly where left off – current_question tells LLM what was last asked, no history needed

Path B -- Fresh start:

SQLite READ → SELECT profile_json FROM user_profile WHERE phone=?

Neo4j WRITE → CREATE new Session node (session_id="s2")

Neo4j WRITE → MERGE each field from profile_json as Attribute nodes

SQLite WRITE → INSERT INTO sessions (session_id="s2", ...)

→ skip rounds already collected, jump to correct workflow_step

Workflow Steps — KKB

Step	Mental State	System Behaviour	Entry Condition
awaiting_consent	FOG	Ask for consent. Present privacy terms.	New user (phone not in SQLite users)
profile_building	ORIENTATION	Collect profile fields round by round.	Consent resolved
market_truth	ORIENTATION	Answer job market / scheme / training queries via RAG.	Profile collection complete
skill_evaluation_a	EVALUATION	Compare profile against ONEST live demand.	User expresses job search intent
skill_evaluation_b	EVALUATION	Deeper assessment for partial-match candidates.	Partial match in skill_evaluation_a
apply_confirmation	COMMITMENT	Single yes/no confirmation turn.	Strong match confirmed
applied	FOLLOW-THROUGH	Application submitted. Handle status queries.	User confirmed in apply_confirmation

Step transitions are driven by NLU intent classification + workflow routing logic in Agent Core. The Memory Layer only stores the current step — it does not own transition logic.

Data Lifecycle — Consent and Retention

Writes happen regardless of consent. Data is always collected during the session so the flow can complete. What differs is what survives after the session ends.

consent_granted=1 (user agreed):

Neo4j graph	→ persists indefinitely (used for returning user resume)
SQLite user_profile	→ written when profile_building completes (used for fresh-start pre-population)
SQLite sessions	→ persists (for returning user lookup)
SQLite turns	→ persists (audit log)

consent_granted=0 (user declined):

Neo4j graph	→ DETACH DELETE at session end – profile knowledge is erased
SQLite user_profile	→ DELETE WHERE phone=? at session end – no cross-session profile
SQLite sessions	→ KEPT – session metadata is audit data, not profile data
SQLite turns	→ KEPT – turn log is audit data, not profile data
SQLite users row	→ KEPT with consent_granted=0 – so system knows on return to re-ask consent

DPDP right-to-erasure (user explicitly requests deletion):

Neo4j	→ DETACH DELETE all Session nodes for this phone
SQLite users	→ DELETE WHERE phone=? – CASCADE wipes sessions, turns, user_profile entirely
This is a full wipe. The users row itself is gone so the phone is unknown on next contact.	

The Trust Layer enforces the right-to-erasure gate. Session-end cleanup for no-consent is handled by the orchestrator after session close, not during the turn.

Domain Agnosticism — How Relationship Types Stay Config-Driven

Neo4j cannot parameterise relationship type names in Cypher — `[:$rel_type]` is invalid syntax.

The solution:

- At startup, Memory Layer reads `profile_graph_relations` from `memory_layer/config/domain.yaml` and builds an internal **whitelist** of allowed

relationship type strings.

- Before any write, it validates the string against the whitelist and regex `^[A-Z][A-Z0-9_]*$`, then builds Cypher with string formatting.
- The orchestrator never handles relationship type strings — it passes only field names.

The Memory Layer Python code contains **zero domain strings**. All relationship type names arrive as runtime values from config.

For a different domain — change the config, the code is untouched.